

0004 — Logan's "MyReversals" System (!PROD_ES_5) — A Code Decode — 2026-06-24

****Series:**** MC Setup Research Notes · Note 0004 ****Confidence:**** Medium — this is a decode and translation, not an edge study. Translation fidelity is ****High**** for the ~half of setups whose logic is fully on-screen, and ****Medium/Low**** where a setup leans on an un-recovered external EasyLanguage function or sat on a partially-scrolled frame. ****No backtest was run for this note — it makes ZERO edge claims.**** The only performance figures quoted are Logan's own //pf source comments, reproduced verbatim and explicitly unverified.

TL;DR: Logan's !PROD_ES_5 is an **intraday ES reversal system on the 5-minute chart**, one position at a time, fixed size, limit-order entries, day-flat. It computes a shared analysis layer (average bar range, two EMAs, a Z-score "big bar" detector, an "Always-In" regime, an Opening-Gap classifier, and an end-of-day "Day Summary") and then fires **17 discrete entry setups (13 long, 4 short)**, all managed by **one 6-rule exit package** (protective stop at 2×ABR, target at 5×ABR, a break-even scratch, a no-progress time bail, and an EOD flatten). This note documents every variable, translates each setup and exit to readable NinjaScript, and—critically—**flags the ~13 external EL helper functions whose source we do NOT have** (GetCC, GetCOLTally, GetAvgBOUp/Down, GetZScoreData, GetBarDir, GetEMASlope/GetAvgSlope, GetABR, GetMidPoint, GetIBS, GetBarRange, GetBarNumber). Half the setups cannot be reproduced 1:1 until those are recovered or re-derived. **Recommendation: treat this as the spec sheet; before porting, re-derive the 4 high-impact black-box functions (§3) — everything else is faithfully translated here.**

1. The system at a glance (premises)

These are read **from the code**, not assumed. Evidence column cites the literal token that fixes each call.

Dimension	Reading	Evidence in the source
Instrument	ES (E-mini S&P 500)	chart tab !PROD_ES_5; GetIBS internal-bar-strength logic
Timeframe	5-minute	strategy name _ES_5; barnum runs 1→78 and Day Summary fires at barnum=78. RTH 08:30–15:00 CT = 390 min ÷ 5 = 78 bars . Decisive.
Session	RTH, day-flat	if Time=835 (first 5-min bar close), barnum=1 resets state, an EOD time-exit at barnum>=76
Direction	Reversal system ("MyReversals")	most setups key off alwaysInDir flips + flipBar; limit entries lean against the last push
Position model	one position at a time	every entry gated Marketposition=0; no add-to-position logic anywhere
Sizing	fixed posSize contracts	identical ... posSize contracts on all 17 entries — no per-setup or volatility sizing
Scaling / pyramiding	none	no scale-in / scale-out on entries; exits act on the whole Currentcontracts block
Entry style	next-bar LIMIT orders	uniformly Buy/Sellshort ... next bar ... at ent limit — never market
Stop / target	2×ABR stop, 5×ABR target (≈2.5:1)	exit package Entryprice-(abr2) stop / Entryprice+(abr5) limit
Style	intraday reversal day-trade (swing-sized target)	scalp-like entries (range-fraction limits), but a 5×ABR ≈ 30–40 pt target — see §1.1
Trade windows	per-setup barnum gates + 3 timers	open-of-day vs middle-of-day packages; stopTradingTimer / stopLongsTimer / stopShortsTimer

Mental model. Through the session the strategy maintains a directional regime (alwaysInDir, ±1) that flips on either a ≥1σ-range bar crossing the fast EMA or two consecutive closes across it. Setups then look for counter-thrust entries against that regime — a strong bar the other way, a gap, a momentum exhaustion (RSI>70), or a specific bar-direction sequence — and place a limit order a fraction of the current bar's range away from price.

A single exit package then manages whatever fills. The crucial point is that the entry and the exit live on different scales: the entry is scalp-like (a limit a few ticks off the close), but the **target is a swing-sized 5×ABR move** held until it pays, stops out, or the session ends. So this is a reversal **day-trade**, not a scalp — the next section quantifies why.

1.1 What 5×ABR actually means (live ES context)

GetABR(20) is a rolling 20-bar average of the bar range, and the exit uses the ABR at the moment of entry. To make the 2×/5× multiples concrete, here is the measured average 5-minute RTH bar range on ES (our data/bars/_continuous.parquet, 2021-06 → 2026-06). This is **descriptive market context, not a backtest** — it sizes the brackets, it does not test them.

Window	ABR (pts)	2×ABR stop	5×ABR target	Target in \$ (1 contract)
2026 YTD	7.7	15.5 pt	38.7 pt	≈ \$1,935
Last 60 sessions	7.6	15.1 pt	37.9 pt	≈ \$1,894
Full 5-yr average	6.1	12.3 pt	30.7 pt	≈ \$1,535

Window	ABR (pts)	2×ABR stop	5×ABR target	Target in \$ (1 contract)
Opening drive (bars 1–6)	8.8	17.7 pt	44.2 pt	≈ \$2,208
Mid-session (bars 30–54)	5.3	10.5 pt	26.3 pt	≈ \$1,314

At today's ABR (~7–8 pts), a 5×ABR target is **~30–40 ES points — roughly half a full RTH session's range, around \$1,500–2,000 per contract**. That is not a scalp target; a scalp would aim for ~1–2×ABR (a handful of ticks). The reward:risk on the bracket is **2.5:1** (5×ABR target over 2×ABR stop), and because the no-progress bail only fires when a trade never trades green, winners are explicitly allowed to run for many bars. The honest one-line style description is therefore intraday reversal day-trade with a swing-sized target.

Completeness legend (used on every setup & exit below)

- [G] **[G] Complete** — entry/exit logic is fully visible and depends only on native EL primitives we can map directly.
- [Y] **[Y] Black-box-gated** — logic is visible but its truth depends on an external GetXxx() function whose source we do not have (behavior inferred from name + usage).
- [R] **[R] Partial** — part of the block was on a scrolled/low-resolution frame; transcription is reconstructed and must be verified against the source.

(In the PDF the coloured dots render as [G] / [Y] / [R].)

2. Variables & the analysis pipeline

2.1 Declared variables (verbatim from the Vars: block)

```
// execution / core
sl(0), tp(0), oneR(0), barnum(0), condTxt(1),
emaFast(0), emaSlow(0), dist(0), mid(0), openF(0), openP(0),
// pattern flags
bullMG(0), bearMG(0),      // micro-gaps (2-bar non-overlap)
bullTG(0), bearTG(0),      // tick-gaps (open vs prior close)
// session timers
stopTradingTimer(0), stopLongsTimer(0), stopShortsTimer(0),
// DAY SUMMARY vars
daySummary(""), bar1Dir(0), bar2Dir(0),
openingGap(0), openingGapClass(""),
dOpen(0), dClose(0), dHigh(0), dLow(0), touchedProfit(0),
// ALWAYS IN vars
flipBar(0), zScore(0), alwaysInDir(0), bou(0), bod(0)
```

range (= High - Low) and posSize (the order size input) are EL reserved/input symbols, not declared here. NinjaScript equivalents:

```
// fields on the strategy
private double sl, tp, oneR, emaFastV, emaSlowV, dist, mid, openP;
private int    barnum, bar1Dir, bar2Dir;
private double openingGap; private string openingGapClass = "", daySummary = "";
private double dOpen, dClose, dHigh, dLow;
private bool   touchedProfit;
private int    flipBar, alwaysInDir;      // alwaysInDir in {-1,+1}
private double zScore, bou, bod;
private int    stopTradingTimer, stopLongsTimer, stopShortsTimer;
private int    posSize = 1;               // Logan's fixed size (input)
// per-bar helpers
private double Range => High[0] - Low[0];
```

2.2 EL → NinjaScript primitive map (applies to every snippet below)

EasyLanguage	NinjaScript	Notes
C, O, H, L / C[1]	Close[0], Open[0], High[0], Low[0] / Close[1]	bar-ago indexing identical
range	High[0] - Low[0]	EL reserved word
XAverage(C,20)	EMA(Close,20)[0]	exponential MA
RSI(C,20)	RSI(Close,20,1)[0]	NT8 RSI takes (period, smooth)
Highest(H,78) / Lowest(L,2)	MAX(High,78)[0] / MIN(Low,2)[0]	
Absvalue(x)	Math.Abs(x)	
Marketposition=0 / >0 / <0	Position.MarketPosition == MarketPosition.Flat / .Long / .Short	
Entryprice	Position.AveragePrice	
Currentcontracts	Position.Quantity	current open size
Barssinceentry	BarsSinceEntryExecution(0, "", 0)	help guide p.2243
Buy(...) next bar at ent limit	EnterLongLimit(posSize, ent, "LONG1")	help guide p.2313
Sellshort(...) next bar at ent limit	EnterShortLimit(posSize, ent, "SHORT1")	help guide p.2320

EasyLanguage	NinjaScript	Notes
Sell(...) at price stop/limit	ExitLongStopMarket(...) / ExitLongLimit(...)	p.2334 / p.2328
Time=835	ToTime(Time[0]) == 83500	first 5-min RTH bar
Text_New(...) / Text_SetColor	Draw.Text(...)	chart annotation only

All order-method **page numbers above are real locations in the NinjaTrader 8 Help Guide** (used to confirm signatures while translating). Verify the exact overload (quantity-first vs price-first) against those pages before compiling.

2.3 Shared analysis pipeline (runs every bar, before any entry)

This is the machinery the setups reference. Documented once here.

(a) Core measures [Y] — GetABR, GetBarNumber, GetMidPoint are black boxes (see §3).

```
abr      = GetABR(20);                // average bar range, ~20 bars
barnum   = GetBarNumber(Time);        // 1..78 within RTH session
emaFast  = XAverage(C,20);
emaSlow  = XAverage(C,200);
mid      = GetMidPoint(false,H,L,0,C); // false => HL2 (inferred)
dist     = Absvalue( GetMidPoint(false,H,L,0,C) - emaFast ); // bar midpoint distance from fast EMA
```

(b) Micro-gaps [G] — two bars that don't overlap.

```
bullMG = 0; bearMG = 0;
if (H[2] < L) then bullMG = 1;    // bar 2 ago entirely below current low
if (L[2] > H) then bearMG = 1;    // bar 2 ago entirely above current high

bool bullMG = High[2] < Low[0];
bool bearMG = Low[2] > High[0];
```

(c) Z-score "big bar" + tick-gaps [Y] (GetZScoreData) / [G].

```
zScore = GetZScoreData(H,L);          // z-score of bar RANGE vs recent (inferred)
bullTG = 0; bearTG = 0;
if (C[1] < 0 - abr*.03) then bullTG = 1;    // open gapped up off prior close
if (C[1] > 0 + abr*.03) then bearTG = 1;    // open gapped down off prior close
```

(d) Opening-Gap classifier [G] (transcription confirmed on the upscaled frame; the logic is suspect — see note).

```
if (barnum = 1) then begin
    openP = Open; dOpen = openP;
end;
if (0 < C[1]) then openingGap = L[1] - H;    // confirmed
if (0 > C[1]) then openingGap = L - H[1];
openingGapClass = "NA";
if (openingGap > abr*1.5) then begin
    if (L[1]-H < 0) then openingGapClass = "BGU";    // "Big Gap Up"
    if (L-H[1] < 0) then openingGapClass = "BGD";    // "Big Gap Down"
end;
```

Used by LONG #1 (openingGapClass <> "NA"). **Verified transcription, but the logic looks internally inconsistent:** $0 < C[1]$ is an open below the prior close (a gap down) yet it feeds the $L[1]-H$ term that, when negative, is tagged **BGU (up)**; the $0 > C[1]$ branch mirrors it for **BGD**. The branch directions and the class labels appear crossed. This is almost certainly a Logan bug (or the labels mean something non-obvious). The practical effect on LONG #1 is small — it only checks $\neq$ "NA", i.e. "a big gap of either sign exists" — but anyone keying long/short off BGU/BGD must resolve this first.

(e) Always-In regime [Y] (GetZScoreData, GetMidPoint) — the directional state.

```
flipBar = 0;
// 1) ONE BIG BAR across the fast EMA
if (zScore >= 1 and mid > emaFast) then begin
    if (alwaysInDir = -1) then flipBar = 1;
    alwaysInDir = 1;
end;
if (zScore >= 1 and mid < emaFast) then begin
    if (alwaysInDir = 1) then flipBar = 1;
    alwaysInDir = -1;
end;
// 2) TWO CONSECUTIVE CLOSES across the fast EMA
if (C > emaFast and C[1] > emaFast) then begin
    if (alwaysInDir = -1) then flipBar = 1;
    alwaysInDir = 1;
end;
if (C < emaFast and C[1] < emaFast) then begin
    if (alwaysInDir = 1) then flipBar = 1;
    alwaysInDir = -1;
end;
```

```

flipBar = 0;
if (zScore >= 1 && mid > emaFastV) { if (alwaysInDir == -1) flipBar = 1; alwaysInDir = 1; }
if (zScore >= 1 && mid < emaFastV) { if (alwaysInDir == 1) flipBar = 1; alwaysInDir = -1; }
if (Close[0] > emaFastV && Close[1] > emaFastV) { if (alwaysInDir == -1) flipBar = 1; alwaysInDir = 1; }
if (Close[0] < emaFastV && Close[1] < emaFastV) { if (alwaysInDir == 1) flipBar = 1; alwaysInDir = -1; }

```

Note: this is the same regime we already ported to NT8 as AlwaysIn.cs in session 36 (handoff S36). That port and this decode agree.

(f) Day Summary (end-of-day classifier, chart label only) [G].

```

if (barnum = 78) then begin
  Value1 = C - openP;
  daySummary = "TR"; // default = Trading Range
  if (Value1 > abr*4) then daySummary = "BL TR"; // bullish trend day
  if ((-Value1) > abr*4) then daySummary = "BR TR"; // bearish trend day
end;
// running day shape
dClose = C; dHigh = Highest(H,78); dLow = Lowest(L,78);

```

daySummary is read by LONG #8 and #12 (daySummary <> "BL TR"). Because it only settles at bar 78 but those setups fire at bars 45–60, they read the **default/partial** value, not the finalized label — worth flagging to Thomas (possible look-ahead-ish quirk in Logan's logic).

(g) Session reset & timer decay [G].

```

if (barnum = 1) then begin
  stopTradingTimer = 0; stopLongsTimer = 0; stopShortsTimer = 0;
end;
if (Marketposition = 0) then touchedProfit = 0;
if (stopTradingTimer > 0) then stopTradingTimer = stopTradingTimer - 1;
if (stopLongsTimer > 0) then stopLongsTimer = stopLongsTimer - 1;
if (stopShortsTimer > 0) then stopShortsTimer = stopShortsTimer - 1;

```

CONFIRMED FINDING (verification pass, all 14 frames): the three timers are only ever **reset to 0** (bar 1) or **decremented** (above). There is **no stopTimer = <positive> assignment anywhere in the provided screenshots**. So as supplied, every stopLongsTimer<=0 / stopTradingTimer<=0 gate is **permanently true** — the cooldown machinery is wired but never armed. Two readings: (a) the arming code (e.g. "after a stop-out, set stopLongsTimer = N") lives in a part of the strategy that was never screenshotted, or (b) it was removed/never finished. Either way, **do not assume a post-loss cooldown exists** — in a faithful port of this source the timers do nothing. Resolve against Logan's full source before relying on it.

3. The black-box functions (the real gap)

These are called throughout but their **bodies are not in any screenshot**. For each: how it's used, our 2–3 best guesses, and a NinjaScript stub. The four bolded ones gate the most setups and should be re-derived first.

Function	Used as	Best guesses (ranked)	Conf
GetABR(n)	abr=GetABR(20); unit for all stops/targets	1. Average Bar Range $\text{mean}(H-L, n) \cdot 2$. ATR(n) · 3. avg body	[G] high
GetBarNumber(Time)	session bar index 1..78	1. bars since session open +1 · 2. bars since a time	[G] high
GetMidPoint(false,H,L,O,C)	mid, dist	1. false→HL2, true→OHLC4 · 2. body mid (O+C)/2	[G] high
GetZScoreData(H,L)	zScore>=1 = "big bar"	1. z-score of range $(\text{range}-\mu)/\sigma$ · 2. z of (H-L)/ATR · 3. z of close	[Y] med
GetBarDir(H,L,O,C)	returns ±1; sequences	1. +1 if C>O else -1 · 2. sign(C-HL2) · 3. IBS>50	[Y] med
GetBarRange(H[1],L[1])	>abr1.5	1. H-L of that bar · 2. true range incl. gap	[G] high
GetIBS(H,L,O,C)	>70/>60/>50/<40	1. Internal Bar Strength $(C-L)/(H-L) \cdot 100$ · 2. same, 0–1	[G] high
GetEMASlope / GetAvgSlope	>0	1. sign of EMA-EMA[n] (rising) · 2. linreg slope · 3. avg-price slope	[Y] med (two distinct fns)
GetCC	>3 (L2), <2 (S1), >2 (S2)	1. consecutive-close count (bars same side of EMA) · 2. candles since flip · 3. close-streak	[R] low
GetCOLTally(n)	>=4 "closing over lows, one bull formed"	1. count in last n bars Closing Over prior Low · 2. close-over-level tally · 3. bull-bar count	[Y] med

Function	Used as	Best guesses (ranked)	Conf
GetAvgBOUp(n) / GetAvgBODown(n)	vs bou/bod	1. avg break-out extension beyond prior bar H/L over n (matches bou=H-H[1], bod=L[1]-L, floored at 0) · 2. avg body-out · 3. avg of bou/bod series	[Y] med
Currentcontracts	exit size	native EL keyword = Position.Quantity	[G] high

Stubs to drop into NinjaScript (replace bodies once re-derived):

```
// [G] high-confidence, safe to implement as-is
double GetABR(int n) { double s=0; for(int k=0;k<n;k++) s+=High[k]-Low[k]; return s/n; }
double GetIBS(double h,double l,double o,double c){ return (h==l)?50:(c-l)/(h-l)*100.0; }
double GetMidPoint(bool ohlc4,double h,double l,double o,double c){ return ohlc4?(o+h+l+c)/4:(h+l)/2; }

// [Y] / [R] ? INFERRED. Verify against Logan's source before trusting.
int GetBarDir(double h,double l,double o,double c){ return c>=o ? 1 : -1; } // TODO confirm
double GetZScoreData(double h,double l){ /* TODO z-score of bar range over N */ return 0; }
int GetCC(){ /* TODO consecutive-close count */ return 0; }
int GetCOLTally(int n){ /* TODO closes-over-prior-low in last n */ return 0; }
double GetAvgBOUp(int n){ /* TODO avg of max(H-H[1],0) over n */ return 0; }
double GetAvgBODown(int n){ /* TODO avg of max(L[1]-L,0) over n */ return 0; }
bool GetEMASlopeUp(){ return EMA(Close,20)[0] > EMA(Close,20)[3]; } // TODO confirm lookahead
```

4. The 17 entry setups

Every entry shares this wrapper; per-setup blocks below show only the **distinguishing predicate** and the entry line.

```
// COMMON WRAPPER (all setups)
if (Position.MarketPosition == MarketPosition.Flat
    && stopLongsTimer <= 0 && stopTradingTimer <= 0
    && barnum > LO && barnum < HI)
{
    if ( <setup predicate> )
    {
        double ent = <entry price>;
        EnterLongLimit(posSize, ent, "LONGn"); // or EnterShortLimit for shorts
    }
}
```

4.1 When each setup is live (session map)

The setups are not a flat list — each is gated to a slice of the session, so the day has a shape. Read top-to-bottom this is roughly: a first-hour gap/reversal cluster, a broad mid-session reversal band, a late-session trend-continuation band, and a handful of always-on momentum/exhaustion plays.

barnum window	approx. CT clock	Setups live	Theme
2–12	08:40–09:30	L1, L10	open-of-day gap / first-two-bar reversal
5–55	09:00–13:05	S1, S2	regime-flip shorts
7–45	09:05–12:15	L5	dynamic-R micro-channel long
10–55	09:20–13:05	L9, L13, S4	exhaustion / regime-flip reversals
12–55	09:30–13:05	L2, L3	mid-session counter-thrust
18–40	10:00–11:50	L6	up-breakout thrust
20–60	10:10–13:30	L4	alternating-bar sequence
45–60	12:15–13:30	L8, L12	late first-bar trend continuation
no gate	any time	L7, L11, S3	RSI>70 momentum / closing-over-lows fade

4.2 Entry aggressiveness (fill behavior)

The entry **price** matters as much as the predicate, because these are limit orders that only fill if price reaches them. Four behaviors:

Class	Entry price	Setups	What it needs to fill
At-market-ish	C or H	L6, L7, L11, S3	almost-immediate; touches on the next bar
Shallow pullback	C – range×.3	L1, L3, L4, L8, L9, L10, L12	a minor retrace into the bar
Deep pullback	C – range×.5 / ×.75	L5, L2	a substantial give-back before entry
Beyond the extreme	L – abrx.25 (long) · C + range, H + abrx.25 (short)	L13, S1, S2, S4	price to push further against you first — "buy a deeper flush / sell into more strength"

The deep-pullback and beyond-the-extreme classes are the truest "reversals": they deliberately sit where price has to extend more before the limit triggers, which both improves entry price and lowers fill rate. A realistic backtest must model these as limits that frequently **do not fill** — counting them as market fills would overstate the edge.

4.3 One position at a time — order is a silent priority

Every entry is gated `Marketposition=0`, and there is no pyramiding. So whenever two or more setups qualify on the same bar (common in the overlapping mid-session band above), **the first one whose limit fills takes the position and blocks all the others** until the trade is closed. That makes the strategy's **top-to-bottom code order an implicit priority ranking** — LONG #1 outranks LONG #13, SHORT #1 outranks SHORT #4. Any port must preserve evaluation order (or make the priority explicit), or it will silently take different trades than Logan's original.

Direction is also lopsided by design: **13 long setups vs 4 short** — a structural long lean consistent with ES's secular drift, though all four shorts are regime/exhaustion fades rather than trend-shorts.

Long setups

LONG #1 — Open-of-day gap, strong first bar · [G] (author note: //pf 1.92) Window bars 2–12; needs a real opening gap, an up first-bar, price over the slow EMA, and a very strong day-level IBS (>70 on the day's O/H/L/C so far).

```
if (Marketposition=0 and stopTradingTimer<=0 and stopLongsTimer<=0
    and barnum>2 and barnum<12 and openingGapClass<>"NA") then
    if (bar1Dir=1 and C>emaSlow and GetIBS(dHigh,dLow,dOpen,dClose)>70) then begin
        ent = C - range*.3; Buy("LONG1") next bar posSize contracts at ent limit;
    end;

if (barnum>2 && barnum<12 && openingGapClass!="NA"
    && bar1Dir==1 && Close[0]>emaSlowV && GetIBS(dHigh,dLow,dOpen,dClose)>70)
    EnterLongLimit(posSize, Close[0]-Range*0.3, "LONG1");
```

LONG #2 — Counter-trend strong-down bar in a down regime · [Y] (GetCC,GetIBS) (author note: //pf 1.52) Mid-day (12–55): an up-close bar with weak IBS (<40) while the regime is down and a consecutive-close count exceeds 3. Fades the down regime with a deep limit ($\text{range} \times 0.75$).

```
if (Marketposition=0 and stopLongsTimer<=0 and barnum>12 and barnum<55) then
    if (C>0 and GetCC>3 and GetIBS(H,L,O,C)<40 and alwaysInDir=-1) then begin
        ent = C - range*.75; Buy("LONG2") next bar posSize contracts at ent limit;
    end;

if (barnum>12 && barnum<55 && Close[0]>Open[0] && GetCC()>3
    && GetIBS(High[0],Low[0],Open[0],Close[0])<40 && alwaysInDir==-1)
    EnterLongLimit(posSize, Close[0]-Range*0.75, "LONG2");
```

LONG #3 — Big bull bar closing near its high · [Y] (GetBarRange,GetIBS,GetBarDir) Prior bar is large ($>1.5 \times \text{ABR}$), up, and closed strong (IBS>50); current bar is up. Continuation-style long.

```
if (Marketposition=0 and stopLongsTimer<=0 and stopTradingTimer<=0 and barnum>=12 and barnum<55) then
    if (GetBarRange(H[1],L[1])>abr*1.5 and C[1]>O[1] and GetIBS(H[1],L[1],O[1],C[1])>50
        and GetBarDir(H,L,O,C)=1) then begin
        ent = C - range*.3; Buy("LONG3") next bar posSize contracts at ent limit;
    end;
```

LONG #4 — Alternating bars, higher low, EMA rising · [Y] (GetBarDir,GetEMASlope) A 4-bar up-down-up-down sequence with $\text{Low} > \text{Low}[1]$ and a rising EMA. ($\text{L} > \text{L}[1]$ read as $\text{Low} > \text{Low}[1]$.)

```
if (Marketposition=0 and stopLongsTimer<=0 and stopTradingTimer<=0 and barnum>20 and barnum<60) then
    if (GetBarDir(H[3],L[3],O[3],C[3])=1 and GetBarDir(H[2],L[2],O[2],C[2])=-1
        and GetBarDir(H[1],L[1],O[1],C[1])=1 and GetBarDir(H,L,O,C)=-1
        and L>L[1] and GetEMASlope>0) then begin
        ent = C - range*.3; Buy("LONG4") next bar posSize contracts at ent limit;
    end;
```

LONG #5 — Dynamic-R, stop outside the 3-bar micro-channel · [Y][R] (sizing-order quirk) Sizes oneR off the 2-bar low and only takes the trade if oneR is "big enough" ($>1.25 \times \text{ABR}$).

```
if (Marketposition=0 and stopLongsTimer<=0 and stopTradingTimer<=0 and barnum>7 and barnum<45) then begin
    ent = C - range*.5;
    tp = ent + oneR; // <-- uses oneR BEFORE it is recomputed (see note)
    sl = ent - oneR;
    oneR = ent - Lowest(L,2); // R = distance from entry to 2-bar low
    if (oneR > abr*1.25) then
        Buy("LONG5") next bar posSize contracts at ent limit;
end;
```

! Logic note for Thomas: tp/sl are assigned from oneR one line before oneR is recomputed, so they use the **previous bar's** R. Either a latent bug or an intentional one-bar lag. Also the $\text{oneR} > \text{abr} \times 1.25$ operator reads as $>$ (sufficient-size filter); double-check against source. [R] because of the scroll + this ordering.

LONG #6 — Up-breakout thrust · [Y] (GetAvgBOUp) Bars 18–40; average up-breakout extension over 15 bars exceeds half an ABR. Enters at the close.


```

if (Marketposition=0 and stopLongsTimer<=0 and stopTradingTimer<=0 and barnum>18 and barnum<40) then
  if (GetAvgBOUp(15)>abr*.5) then begin
    ent = C; Buy("LONG6") next bar posSize contracts at ent limit;
  end;

```

LONG #7 — Momentum (RSI>70) above both EMAs · [G] (RSI native) Price over the session open, low holding above the fast EMA, fast>slow EMA, RSI(20)>70.

```

if (Marketposition=0 and stopLongsTimer<=0 and stopTradingTimer<=0
  and C>openP and L>emaFast and emaFast>emaSlow and RSI(C,20)>70) then begin
  ent = C; Buy("LONG7") next bar posSize contracts at ent limit;
end;

```

Near-identical to **LONG #11** (which drops the timer gates). Treat #7/#11 as one idea in two windows.

LONG #8 — Late-session first-bar trend, not on a bull-trend day · [Y][R] (daySummary timing) Bars 45–60; up first-bar, price over slow EMA, and daySummary <> "BL TR".

```

if (Marketposition=0 and stopLongsTimer<=0 and stopTradingTimer<=0
  and barnum>45 and barnum<60 and daySummary<>"BL TR") then
  if (bar1Dir=1 and C>emaSlow) then begin
    ent = C - range*.3; Buy("LONG8") next bar posSize contracts at ent limit;
  end;

```

daySummary only finalizes at bar 78, so here it reads its default/partial value — see §2.3(f).

LONG #9 — "Big-diff" exhaustion reversal · [Y] (GetAvgBOUp/Down,GetBarDir,GetAvgSlope) (author notes //.02pf, //big diff .1 pf) Strong up-breakout average, tiny down-breakout average, up regime, prior bar up but current bar down, above slow EMA, rising avg slope. The annotations are Logan's per-clause marginal PF contributions.

```

if (Marketposition=0 and barnum>10 and barnum<55) then
  if (GetAvgBOUp(5)>.4 and GetAvgBODown(5)<.1 // big diff
    and alwaysInDir=1 and C>openP // .02 pf
    and GetBarDir(H[1],L[1],O[1],C[1])=1
    and GetBarDir(H,L,O,C)=-1 // big diff .1 pf
    and C>emaSlow and GetAvgSlope>0) then begin
    ent = C - range*.3; Buy("LONG9") next bar posSize contracts at ent limit;
  end;

```

LONG #10 — Open-of-day reversal first-two bars · [Y] (GetIBS,GetBarDir) Bars 2–12; up first-bar then down second-bar, over slow EMA, day-level IBS>50.

```

if (Marketposition=0 and barnum>2 and barnum<12) then
  if (bar1Dir=1 and bar2Dir=-1 and C>emaSlow and GetIBS(dHigh,dLow,dOpen,dClose)>50) then begin
    ent = C - range*.3; Buy("LONG10") next bar posSize contracts at ent limit;
  end;

```

LONG #11 — Momentum (RSI>70), no time gate · [G] (RSI native)

```

if (Marketposition=0 and C>openP and L>emaFast and emaFast>emaSlow and RSI(C,20)>70) then begin
  ent = C; Buy("LONG11") next bar posSize contracts at ent limit;
end;

```

Duplicate-family with **LONG #7**.

LONG #12 — Late first-bar trend (no timers) · [G]

```

if (Marketposition=0 and barnum>45 and barnum<60 and daySummary<>"BL TR") then
  if (bar1Dir=1 and C>emaSlow) then begin
    ent = C - range*.3; Buy("LONG12") next bar posSize contracts at ent limit;
  end;

```

Duplicate-family with **LONG #8** (minus the timer gates). Same daySummary timing caveat.

LONG #13 — Regime-flip reversal off a small down bar · [Y] (GetAvgBODown) Down regime just flipped (alwaysInDir=-1 and flipBar=1), a small down-extension bar (bod < ½·avg), a mid-sized range (½–1 ABR), above slow EMA. Limit below the low (L - abr.25).

```

if (Marketposition=0 and barnum>10 and barnum<55) then
  if (alwaysInDir=-1 and flipBar=1 and bod<GetAvgBODown(10)*.5
    and range>abr*.5 and range<abr and C>emaSlow) then begin
    ent = L - abr*.25; Buy("LONG13") next bar posSize contracts at ent limit;
  end;

```

Short setups

SHORT #1 — Up-regime flip, strong-IBS up bar · [Y] (GetIBS,GetCC) Just-flipped to up regime, strong bar (IBS>50), up close, low consecutive-close count (<2). Fades into a C+range limit.

```

if (Marketposition=0 and barnum>5 and barnum<55) then

```

```
if (flipBar=1 and GetIBS(H,L,O,C)>50 and alwaysInDir=1 and C>0 and GetCC<2) then begin
  ent = C + range; Sellshort("SHORT1") next bar posSize contracts at ent limit;
end;
```

```
if (barnum>5 && barnum<55 && flipBar==1 && GetIBS(High[0],Low[0],Open[0],Close[0])>50
  && alwaysInDir==1 && Close[0]>Open[0] && GetCC()<2)
  EnterShortLimit(posSize, Close[0]+Range, "SHORT1");
```

SHORT #2 — Large up bar, strong IBS, down regime, far from EMA · [Y] (GetCC,GetIBS) Up bar that is large (range>ABR) or has a high consecutive-close count, IBS>60, regime down, and bar midpoint far (>½ ABR) from the fast EMA.

```
if (Marketposition=0 and barnum>5 and barnum<55) then
  if (C>0 and (range>abr*1 or GetCC>2) and GetIBS(H,L,O,C)>60
    and alwaysInDir=-1 and dist>abr*.5) then begin
    ent = C + range; Sellshort("SHORT2") next bar posSize contracts at ent limit;
  end;
```

SHORT #3 — "Closing over lows, one bull formed" · [Y] (GetCOLTally) — **confirmed unguarded** At least 4 of the last 5 bars closed over prior lows while the regime is up — an exhausted up push to fade at the high.

```
if (GetCOLTally(5))>=4 and alwaysInDir=1) then begin // been closing over lows, one bull formed
  ent = H; Sellshort("SHORT3") next bar posSize contracts at ent limit;
end;
```

CONFIRMED (verification pass): the block is fully visible top-and-bottom and genuinely has **no Marketposition=0 and no barnum window** — it is the **only un-gated setup** in the system. Consequences a port must handle: (1) it can fire **at any time of day**, including bar 1 or the final bars; (2) with no flat-gate, a Sellshort while already **long** will, under TradeStation defaults, **reverse the position** (close the long and go short) rather than be ignored — so SHORT #3 can flip an open long trade. This may be intentional (an exhaustion override) or an oversight. In NinjaScript, replicate the exact reverse-vs-ignore behavior deliberately; do not just bolt on a flat-gate, which would change Logan's logic.

SHORT #4 — Regime-flip reversal off a small up bar (mirror of LONG #13) · [Y] (GetAvgBOUp) Up regime just flipped, small up-extension bar, mid range, below slow EMA. Limit above the high.

```
if (Marketposition=0 and barnum>10 and barnum<55) then
  if (alwaysInDir=1 and flipBar=1 and bou<GetAvgBOUp(10)*.5
    and range>abr*.5 and range<abr and C<emaSlow) then begin
    ent = H + abr*.25; Sellshort("SHORT4") next bar posSize contracts at ent limit;
  end;
```

Setup index (author PF annotations are Logan's, UNVERIFIED)

#	Side	One-line	Completeness	Author //pf
1	L	Open gap + strong first bar	[G]	1.92
2	L	Fade strong-down bar in down regime	[Y]	1.52
3	L	Big bull bar closing near high	[Y]	—
4	L	Alternating bars, higher low, EMA up	[Y]	—
5	L	Dynamic-R, stop outside 3-bar MC	[Y][R]	—
6	L	Up-breakout thrust	[Y]	—
7	L	RSI>70 above both EMAs	[G]	—
8	L	Late first-bar trend, not BL-TR day	[Y][R]	—
9	L	"Big-diff" exhaustion reversal	[Y]	.02 / .1 (marginal)
10	L	Open reversal, up-then-down first two	[Y]	—
11	L	RSI>70 (no time gate) — dup of #7	[G]	—
12	L	Late first-bar trend (no timers) — dup of #8	[G]	—
13	L	Regime-flip reversal off small down bar	[Y]	—
1	S	Up-flip strong-IBS up bar	[Y]	—
2	S	Large up bar far from EMA, down regime	[Y]	—
3	S	Closing-over-lows exhaustion (un-gated; can reverse a long)	[Y]	—
4	S	Regime-flip off small up bar (mirror of L#13)	[Y]	—

5. The exit package (one manager for all 17 entries)

Read at full resolution from the LONG side; the SHORT side mirrors it but part of the short block was off-screen ([R] where noted).

```
// ===== LONG EXITS (Marketposition > 0) =====
if (Marketposition>0) then begin

  // (1) PROTECTIVE STOP ? 2x ABR below entry
  Sell("oats-stop") next bar Currentcontracts contracts at Entryprice-(abr*2) stop;
```



```
// (2) PROFIT TARGET ? 5x ABR above entry (=> ~2.5:1 vs the 2x stop)
Sell("top") next bar Currentcontracts contracts at Entryprice+(abr*5) limit;

// (3) BREAK-EVEN SCRATCH ? went green then came back; leave ~flat
if (C>Entryprice and touchedProfit=1) then
    Sell("MGHTL") next bar Currentcontracts contracts at Entryprice+abr*.1 limit;
if (C<Entryprice) then touchedProfit=1; // <-- see naming note

// (4) NO-PROGRESS TIME BAIL ? open >5 bars, never profitable
if (Barssinceentry>5 and touchedProfit=0) then
    Sell("MGHTL") next bar Currentcontracts contracts at H limit;

// (5) EOD TIME EXIT ? flatten near the close
if (barnum>=76) then
    Sell("EoD") next bar Currentcontracts contracts at C limit;
end;

// ===== SHORT EXITS (Marketposition < 0) =====
if (Marketposition<0) then begin
    // (6) SAFETY STOP ? 1x ABR above entry <-- THE LAST LINE IN THE SCREENSHOTS
    Buytocovert("oats-stop5") next bar posSize contracts at Entryprice+abr*1 stop;
end; // (the source image ends here)
```

CONFIRMED (verification pass): the final screenshot ends immediately after this short safety-stop line. **The short side has no visible profit target, break-even scratch, or time/EOD exit** — only a 1×ABR stop. Two possibilities, and they matter a lot: (a) the rest of the short manager scrolled off the bottom of the last frame (most likely), or (b) shorts genuinely run with **only a stop and no target/time-out**, which would be a severe long/short asymmetry (longs get a 5×ABR target + scratch + EOD flatten; shorts get nothing but a stop and would only close on the stop or a reversing signal). **This is the single most important unresolved gap for anyone trading the short side** — it cannot be settled from the screenshots and needs Logan's source. Until then, treat short exits as unknown beyond the safety stop.

NinjaScript translation of the long manager:

```
if (Position.MarketPosition == MarketPosition.Long)
{
    double entry = Position.AveragePrice;
    ExitLongStopMarket(Position.Quantity, entry - 2*abr, "oats-stop", ""); // (1) p.2334
    ExitLongLimit (Position.Quantity, entry + 5*abr, "top", ""); // (2) p.2328

    if (Close[0] > entry && touchedProfit) // (3)
        ExitLongLimit(Position.Quantity, entry + 0.1*abr, "scratch", "");
    if (Close[0] < entry) touchedProfit = true;

    if (BarsSinceEntryExecution(0,"",0) > 5 && !touchedProfit) // (4)
        ExitLongLimit(Position.Quantity, High[0], "nobail", "");

    if (barnum >= 76) // (5)
        ExitLongLimit(Position.Quantity, Close[0], "EoD", "");
}
```

Exit observations for Thomas:

- **Bracket size in real terms:** at the current ES ABR (~7.7 pt, \$1.1) this stop/target pair is roughly a **15-point stop and a 38-point target** (~\$775 risk for ~\$1,935 reward per contract) — a swing-sized intraday move, not a scalp. The 5×ABR target alone is about half a typical RTH range.
- **Asymmetric stops (confirmed):** longs risk 2×ABR, the short safety stop is 1×ABR — verified, not a transcription slip. Combined with the missing short target/time-exit (above), the long and short sides are managed very differently.
- **touchedProfit naming vs use:** it is set to 1 when **C < Entryprice** (price went against a long), yet rule (3) calls it "closed over entry, lock in BE." The variable behaves more like "has traded below entry at least once" than "touched profit." Logic works, the name is misleading — confirm intent.
- **Target/stop are ABR-multiples, not the oneR from LONG #5.** Only LONG #5 computes oneR; the exit manager ignores it and uses abr2 / abr5 for every setup. So LONG #5's tp/sl math is effectively dead code under this exit package (another reason to review #5).
- **Rule labels** (oats-stop, top, MGHTL) are Logan's order tags; they carry no logic.

6. What we can / can't trust

- **Premises (§1):** High confidence — every row is pinned to a literal token.
- **Pipeline & exits (§2, §5):** High where [G]. The verification pass upgraded the Opening-Gap classifier to confirmed

transcription (§2.3d) and resolved three items from "maybe off-screen" to confirmed findings (timer arming, SHORT #3's missing gate, the short-exit cutoff — all in §6.1).

- **Setups (§4):** the entry logic is fully transcribed for all 17; what remains soft is (a) the black-box functions they call and (b) the handful of logic quirks. Even the "native" setups read author-defined daySummary/openingGapClass produced by [Y]/buggy pipeline code.
- **The single biggest risk** is the meaning of **GetCC** (gates L#2, S#1, S#2) and **GetZScoreData** (gates the entire Always-In regime, hence ~every regime-aware setup). Get these two wrong and the system behaves differently even though the visible code is "complete."

6.1 Open items after the verification pass

Everything below was checked against all 14 frames (several upscaled 2.6–3×). "Closed" = settled from the screenshots; "Blocked" = needs Logan's source and cannot be resolved here.

Open item	Status	Impact
External GetXxx() function bodies (§3)	Blocked — no source	High
Short-exit logic past the 1×ABR stop (§5)	Blocked — frame cut off	Severe
GetCC / GetZScoreData definitions	Blocked	High
Timer arming stopTimer=N (§2.3g)	Closed — confirmed absent; gates always-on	Medium
SHORT #3 has no flat/time gate	Closed — un-gated; can reverse a long	Medium
Opening-Gap BGU/BGD labels crossed (§2.3d)	Closed transcription; logic suspect	Low/High
LONG #5 stale oneR, dead under ABR exits	Closed — ordering quirk	Low
daySummary partial-read at bars 45–60 (§2.3f)	Closed — timing quirk	Medium
touchedProfit set on C<Entryprice (§5)	Closed — works, name misleads	Low

(Blocked = needs Logan's source; Closed = settled from the screenshots. "Severe"/High/Medium/Low = impact on a faithful port.)

Bottom line on completeness: as a decode of the material we were given, the note is now complete — every visible line is transcribed and every gap is explicitly classified Closed or Blocked. It is **not** a turnkey build spec, and cannot be, because items 1–3 (especially the **short-exit logic** and the two regime functions) live in source we don't have.

7. Recommendation & next steps

Treat this note as the build spec — but close the two Blocked gaps before writing a line of production code. Ordered by what unblocks the most:

A. Get from Logan (the only things this note can't supply): 1. **The short-exit block** (open item #2) — the single highest-value ask. Without it the short side is untradeable as a faithful port; one screenshot of the lines below the safety stop settles it. 2. **The function source** for GetZScoreData, GetCC, GetCOLTally, GetAvgBOUp/Down, GetBarDir, GetEMASlope/GetAvgSlope (open items #1, #3). These gate the regime and ~12 setups. 3. **The timer-arming code**, if any (open item #4) — confirm whether a post-loss cooldown was ever written.

B. Do on our side, now (no Logan needed): 4. **Re-derive the 4 tractable functions** — GetABR, GetMidPoint, GetIBS are done as [G] stubs (§3); make and record a decision for GetZScoreData (z-score of bar range over N) and GetCC (consecutive-close count) so the port is runnable with explicit assumptions flagged. 5. **Build the NinjaScript skeleton** with the confirmed pieces: the analysis pipeline (§2.3), the long exit manager (§5), the common entry wrapper (§4), and the session/priority structure (§4.1–4.3). Leave the Blocked functions as the §3 stubs. 6. **Collapse the duplicates** — L#7≈L#11 and L#8≈L#12 are one idea in two windows; implement once, parameterize the gate. 7. **Encode the confirmed quirks deliberately** — the always-on timers (#4), SHORT #3's reverse-on-signal (#5), the daySummary partial read (#8) — replicate, don't "fix," or you change the system.

C. Only after A+B — and in a separate note: 8. Validate the functions empirically (do our re-derived GetCC/GetZScore reproduce Logan's behavior on shared bars?), then run an edge study. **This note asserts no edge.** Logan's //pf 1.92 / 1.52 are author annotations on unknown data and costs and must not be cited as results.

8. Reproduce / provenance

- **Source:** Google Drive folder "Logans system code" (owner tdeutschmann@gmail.com), 14 PNG screenshots of the TradeStation strategy IPROD_ES_5, uploaded 2025-10-30; also Logans system code.zip (same PNGs, no text source).
- **Transcription:** screenshots read directly; the low-resolution exit-package, open-of-day, and short-entry frames were upscaled 2.6–3× (Lanczos) to transcribe exactly. No .eld/text source exists, so all EL here is OCR-from-image and carries the completeness flags above.
- **Verification pass:** a second sweep re-checked all 14 frames specifically for timer-arming assignments, the full short-exit block, and SHORT #3's wrapper. Results are logged in §6.1 (timer-arming and SHORT #3 confirmed; short-exit logic confirmed cut off at the safety stop).

- **NT8 API check:** order/bar methods verified against the NinjaTrader 8 Help Guide PDFs (EnterLongLimit p.2313, EnterShortLimit p.2320, ExitLongLimit p.2328, ExitLongStopMarket p.2334, BarsSinceEntryExecution p.2243, BarsSinceNewTradingDay p.1488).
- **Related internal work:** the Always-In regime here is the same one ported as AlwaysIn.cs and tested (as a gate, negative) in handoff session 36; this note supersedes those scattered notes as the full decode.
- **No scripts / no sims were run for this note** — it is a static code decode.